ASP.NET Core MVC - Cheat Sheet

Xenofon Gkioka | Week 2 Revision

MVC Architecture

Letter	Stands For	Responsibility
M	Model	Data shape — maps to DB table
V	View	UI — renders HTML using Razor
C	Controller	Logic — receives request, talks to DB, returns View

---

Request Flow (URL → Page)

```
User types /Category/Index
→ Router maps to CategoryController.Index()
→ Controller calls _db.Categories.ToList()
→ Controller passes data to View via return View(data)
→ View renders HTML and sends back to browser
```

Program.cs Responsibilities

- Builds the application

- Registers services in DI container (DbContext, MVC)
- Sets up middleware pipeline (HTTPS, routing, auth, static files)
- Defines default route pattern: {controller=Home}/{action=Index}/{id?}
- Runs the application

Dependency Injection

- You don't create objects with `new` — you ask for them
- Register in Program.cs: `builder.Services.AddDbContext<ApplicationDbContext>(...)`
- Inject in constructor: `public CategoryController(ApplicationDbContext db)`
- ASP.NET creates and manages the object lifetime
- Benefits: one place to configure, easy to test, no memory leaks

---

DbContext vs DbSet

	What it is
DbContext	Bridge between C# and the database — manages connection and operations
DbSet<T>	Represents one table — `DbSet<Category>` = Categories table

Migrations

Command	What it does
`dotnet ef migrations add <Name>`	Generates migration file (C# describing SQL changes)
`dotnet ef database update`	Executes the SQL against the database

Inside every migration:

- `Up()` → applies the change
- `Down()` → rolls it back

Data Annotations

Annotation	What it does
`[Key]`	Marks property as Primary Key
`[Required]`	Field cannot be null in DB
`[Range(min, max)]`	Validates numeric range
`[MaxLength(n)]`	Limits string length
`[DisplayName("X")]`	Label shown in UI

Controller Actions

```csharp
// GET - return view
public IActionResult Index() { return View(data); }

// GET - return view with item by id
public IActionResult Edit(int? id) { return View(item); }

// POST - receive form data
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Category obj) {
    if (ModelState.IsValid) {
        _db.Categories.Update(obj);
        _db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(obj);
}
```

EF Core CRUD Operations

```csharp
```

```
_db.Categories.ToList()       // SELECT all
_db.Categories.Find(id)       // SELECT by PK
_db.Categories.Add(obj)       // INSERT
_db.Categories.Update(obj)    // UPDATE
_db.Categories.Remove(obj)    // DELETE
_db.SaveChanges()             // COMMIT — must call after every change
```

Tag Helpers vs HTML

Tag Helper	Generated HTML
`asp-controller="Category" asp-action="Index"`	`href="/Category/Index"`
`asp-for="Name"`	`name="Name" id="Name"` + validation
`asp-route-id="@item.Id"`	`href="/Category/Edit/5"`
`asp-action="Create"` on form	`action="/Category/Create"`

Razor View Syntax

```cshtml
```

```
@model Category                     // declare model type
@model IEnumerable<Category>        // for lists
@item.Name                          // output property
@foreach
(
var
 item
in
 Model
)

{

}
    // loop
@
{
 ViewData
[
"Title"
]

=

"Page"
;

}
   // set title
@* comment *@                       // Razor comment
```

Connection String (appsettings.json)

```json
"ConnectionStrings": {
  "DefaultConnection": "Server=localhost,1433;Database=MyDb;
  User Id=sa;Password=YourPass;TrustServerCertificate=True;"
}
```